2026-7-6 Linux开发工具--实战：进度条代码解

- 1. 进度条实战要解决什么问题
  - 1.1 为什么从进度条开始
    - 1.1.1 一个看似简单的小程序
    - 1.1.2 本节目标
- 2. 回车与换行
  - 2.1 两个动作
    - 2.1.1 回车
    - 2.1.2 换行
  - 2.2 为什么进度条使用 \r
  - 2.3 覆盖时为什么需要固定宽度
- 3. 标准输出与缓冲区
  - 3.1 为什么输出不一定立刻出现
    - 3.1.1 缓冲区在内存中
    - 3.1.2 行缓冲与完全缓冲
  - 3.2 程序结束时会自动刷新
    - 3.2.1 正常退出
    - 3.2.2 异常终止
  - 3.3 三个标准流
    - 3.3.1 stdin、stdout、stderr
    - 3.3.2 Linux 下一切皆文件
  - 3.4 printf 为什么显示字符
- 4. 使用 fflush 强制刷新
  - 4.1 fflush 的作用
  - 4.2 三个对照实验
    - 4.2.1 带换行
    - 4.2.2 不带换行
    - 4.2.3 不带换行但主动刷新
  - 4.3 为什么进度条必须频繁刷新
  - 4.4 不要对输入流随意使用 fflush
- 5. 倒计时程序
  - 5.1 目标
  - 5.2 代码
  - 5.3 关键点
    - 5.3.1 \r 控制回行首
    - 5.3.2 %-2d 固定宽度

        - 5.3.3 fflush 立即显示
        - 5.3.4 sleep 控制节奏
        - 5.3.5 循环结束后换行
    - 5.4 为什么倒计时是进度条的前置练习
- 6. 时间控制：sleep 与 usleep
    - 6.1 sleep 与 usleep 的单位
    - 6.2 进度条为什么不用 sleep(1)
    - 6.3 usleep 的可移植性
- 7. 第一版进度条
    - 7.1 设计目标
    - 7.2 代码
    - 7.3 为什么数组长度是 101
    - 7.4 为什么先打印再填充
    - 7.5 固定宽度格式
    - 7.6 第一版的局限
- 8. 第二版进度条：显示与业务分离
    - 8.1 设计目标
    - 8.2 进度百分比计算
    - 8.3 单独声明显示接口
    - 8.4 实现显示函数
    - 8.5 使用静态旋转索引
    - 8.6 主程序提供真实进度
    - 8.7 第二版为什么更适合真实项目
        - 8.7.1 进度显示与业务分离
        - 8.7.2 代码可复用
        - 8.7.3 更容易维护
- 9. 回调函数与解耦
    - 9.1 直接调用与回调
        - 9.1.1 直接调用
        - 9.1.2 使用函数指针
    - 9.2 回调示例
    - 9.3 回调带来的好处
        - 9.3.1 降低耦合
        - 9.3.2 提高可测试性
        - 9.3.3 保留上层控制
- 10. 边界情况与质量改进
    - 10.1 百分比超界

    - 10.2 total 为 0
    - 10.3 文本残留
    - 10.4 刷新过慢或过快
        - 10.4.1 过慢
        - 10.4.2 过快
    - 10.5 输出被重定向
    - 10.6 中断与输出完整性
- 11. 完整实验流程
    - 11.1 实验一：观察行缓冲
    - 11.2 实验二：倒计时
    - 11.3 实验三：第一版进度条
    - 11.4 实验四：第二版模块化
    - 11.5 实验五：回调接口
- 12. 代码速查与常见错误
    - 12.1 核心技术速查
    - 12.2 常见错误
        - 12.2.1 忘记 fflush
        - 12.2.2 使用 \n 代替 \r
        - 12.2.3 数组越界
        - 12.2.4 百分比没有限制
        - 12.2.5 total 为 0
        - 12.2.6 业务逻辑写死在显示函数中
    - 12.3 学习主线

1. 进度条实战要解决什么问题

1.1 为什么从进度条开始

1.1.1 一个看似简单的小程序

进度条表面上只是不断打印：

```
[=====                ][25%]
```

但它同时涉及多个基础知识点：

- 回车符 `\r` 与换行符 `\n`。
- 标准输出缓冲区。
- `fflush` 强制刷新。
- 固定宽度字符串。
- 百分比与进度计算。
- 循环和延时。
- 函数拆分。
- 回调与解耦。

因此，进度条很适合作为第一个综合性的 Linux 小程序。

1.1.2 本节目标

最终要实现的能力是：

```
程序计算当前进度
→ 进度显示函数接收当前值和总量
→ 同一行不断覆盖旧进度
→ 实时显示百分比、进度条和旋转光标
```

本节只整理进度条相关内容，不展开 Git、Makefile 或其他开发工具。

2. 回车与换行

2.1 两个动作

2.1.1 回车

回车符：

```
\r
```

表示把光标移动到当前行的行首。

进度条需要不停覆盖同一行，因此大量使用 `\r`。

2.1.2 换行

换行符：

```
\n
```

表示把光标移动到下一行。

通常说“回车换行”是两个动作的组合：

```
\r   回到行首
\n   移动到下一行
```

在老式打字机中，回车是把打印头移回行首，换行是把纸张移动到下一行。终端中的 `\r` 和 `\n` 借用了这两个概念。

2.2 为什么进度条使用 \r

如果每一帧都使用 `\n`：

```
[=         ][10%]
[==        ][20%]
[===       ][30%]
```

终端会一直向下输出，不能形成进度条。

如果每帧末尾使用 `\r`：

```
[=         ][10%]\r
[==        ][20%]\r
[===       ][30%]\r
```

下一帧会从行首继续打印，覆盖上一帧的部分内容，看起来就是一个不断变化的进度条。

2.3 覆盖时为什么需要固定宽度

假设上一帧是：

```
100%
```

下一帧变成：

```
5%
```

如果没有固定宽度，上一帧多出来的字符可能残留在屏幕上。

进度条通常使用固定宽度，例如：

```
printf("[%-100s][%3d%%]\r", buffer, percent);
```

这样即使百分比长度变化，整帧的总宽度也基本稳定。

3. 标准输出与缓冲区

3.1 为什么输出不一定立刻出现

3.1.1 缓冲区在内存中

C 标准输入输出库通常会先在内存中准备输出数据，再批量写入实际设备。这个中间区域就是缓冲区。

程序执行：

```
printf("hello");
```

字符串可能已经进入 `stdout` 缓冲区，但还没有立刻显示到终端。

3.1.2 行缓冲与完全缓冲

常见的终端行为是行缓冲：

```
遇到换行
或缓冲区接近满
→ 刷新到终端
```

但标准 C 的缓冲行为取决于运行环境和标准流连接对象。输出被重定向到普通文件时，可能采用完全缓冲，不能简单假设每次 `printf` 都会立即显示。

3.2 程序结束时会自动刷新

3.2.1 正常退出

程序从 `main` 返回或正常调用 `exit` 时，标准流缓冲区通常会被刷新。

因此：

```c
printf("hello");
sleep(3);
return 0;
```

可能先等待 3 秒，然后程序退出时才看到输出。

3.2.2 异常终止

如果程序被 `_exit`、信号等方式异常终止，普通标准流缓冲区不一定会可靠刷新。不能把异常退出的刷新当作正确设计。

3.3 三个标准流

3.3.1 stdin、stdout、stderr

程序启动时通常默认打开三个标准流：

流	文件描述符	常见设备
`stdin`	0	键盘或输入重定向
`stdout`	1	终端或输出重定向
`stderr`	2	终端或错误重定向

3.3.2 Linux 下一切皆文件

Linux 把终端、键盘、普通文件等资源统一抽象成文件。`stdin`、`stdout`、`stderr` 是 C 标准库对这些底层文件的封装。

程序本质上处理的是数据，而数据的来源和去向可以是：

- 用户键盘输入。
- 普通文件。
- 管道。
- 网络连接。
- 显示器。

3.4 printf 为什么显示字符

终端最终显示的是字符。整数、浮点数等数据必须先转换成文本：

```
整数 123
→ printf 格式化
→ 字符串 "123"
→ 写入 stdout
→ 显示到终端
```

因此 `printf` 的格式化输出本质上是在把非字符数据转换成字符序列。

ASCII 等字符编码规定了字符与数值之间的对应关系。

4. 使用 fflush 强制刷新

4.1 fflush 的作用

函数声明：

```c
int fflush(FILE *stream);
```

调用：

```
fflush(stdout);
```

会要求标准输出流立即把缓冲区中的数据写到对应输出设备。

它不会修改字符串内容，只是控制“数据从缓冲区送出”的时机。

4.2 三个对照实验

4.2.1 带换行

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("hello\n");
    sleep(3);
    return 0;
}
```

终端连接下的行缓冲通常会因为换行而立即显示 `hello`，然后再等待 3 秒。

4.2.2 不带换行

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("hello");
    sleep(3);
    return 0;
}
```

可能先等待 3 秒，然后在程序退出刷新缓冲区时看到输出。

4.2.3 不带换行但主动刷新

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("hello");
    fflush(stdout);
    sleep(3);
    return 0;
}
```

这次会先看到 `hello`，然后等待 3 秒。

4.3 为什么进度条必须频繁刷新

进度条在循环中不断生成新帧：

```c
printf("[%-100s][%d%%]\r", buffer, percent);
fflush(stdout);
```

如果不刷新，缓冲区中的数据可能积累很久，用户看到的进度条会卡顿、延迟或直到程序结束才出现。

`fflush(stdout)` 是进度条实时显示的关键。

4.4 不要对输入流随意使用 fflush

`fflush(stdin)` 并不是可移植的“清空输入”方式。标准 C 主要定义了输出流的 `fflush` 行为，输入流处理应使用语言允许的方式。

5. 倒计时程序

5.1 目标

让数字在同一行从 10 显示到 0：

```
count is: 10
count is: 9
```

```
count is: 8
...
count is: 0
```

但屏幕上只占一行，数字不断被覆盖。

5.2 代码

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    int count = 10;

    while (count >= 0)
    {
        printf("count is: %-2d\r", count);
        fflush(stdout);
        sleep(1);
        --count;
    }

    printf("\n");
    return 0;
}
```

5.3 关键点

5.3.1 \r 控制回行首

每次打印结束后，光标回到行首，下一次 `printf` 覆盖上一帧。

5.3.2 %-2d 固定宽度

`%-2d` 让一位数和两位数都占两个字符宽度，减少旧字符残留。

5.3.3 fflush 立即显示

每帧写完后立即刷新，用户才能看到数字按秒变化。

5.3.4 sleep 控制节奏

```
sleep(1);
```

参数单位是秒。常见的 `sleep` 函数来自 `<unistd.h>`。

5.3.5 循环结束后换行

最后：

```
printf("\n");
```

让后续终端提示符出现在下一行，而不是继续覆盖倒计时。

5.4 为什么倒计时是进度条的前置练习

倒计时已经包含进度条的三个核心机制：

```
同一行覆写
+ 固定宽度
+ 主动刷新
```

只需要把“数字进度”扩展成“进度条图形 + 百分比 + 动画”，就得到第一版进度条。

6. 时间控制：sleep 与 usleep

6.1 sleep 与 usleep 的单位

函数	参数单位	常见场景
`sleep`	秒	倒计时
`usleep`	微秒	快速更新进度条

换算关系：

```
1 秒 = 1000 毫秒
1 毫秒 = 1000 微秒
1 秒 = 1,000,000 微秒
```

所以：

```
usleep(50000);
```

表示暂停约：

```
50000 微秒 = 0.05 秒
```

6.2 进度条为什么不用 sleep(1)

如果每增加 1% 都等待 1 秒，完整进度条需要约 100 秒，交互体验太慢。

使用较小延时可以让进度变化看起来连续，同时仍能观察刷新过程。

6.3 usleep 的可移植性

`usleep` 常见于 POSIX 环境，但在一些新标准中已不再推荐。现代代码还可以使用 `nanosleep` 等接口。

学习进度条时使用 `usleep` 便于观察，因为参数直接以微秒表示。

7. 第一版进度条

7.1 设计目标

第一版用于理解原理：

```
[===================================...][100%][|]
```

需要包含：

- 100 个字符长度的进度区域。
- 当前百分比。
- 旋转光标。
- 同一行刷新。
- 固定更新间隔。

7.2 代码

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define NUM 101
#define STYLE '='

void process_v1(void)
{
    char buffer[NUM];
    const char *label = "|/-\\";
    int spin_index = 0;
    int count = 0;

    memset(buffer, 0, sizeof(buffer));

    while (count <= 100)
    {
        printf("[%-100s][%3d%%][%c]\r",
               buffer,
               count,
               label[spin_index % 4]);
        fflush(stdout);

        buffer[count] = STYLE;
        ++count;
        ++spin_index;

        usleep(50000);
    }

    printf("\n");
}
```

```c
int main(void)
{
    process_v1();
    return 0;
}
```

7.3 为什么数组长度是 101

进度从 `0` 到 `100`，最多需要 100 个可见字符：

```
索引 0 到 99 → 100 个字符
索引 100     → 字符串结束符 '\0'
```

所以字符数组至少需要 101 个元素。

如果使用：

```c
char buffer[100];
```

在最后一次写入时可能没有空间保存 `'\0'`，`printf("%s")` 可能继续读取数组后面的内存，产生未定义行为。

7.4 为什么先打印再填充

第一次循环时 `buffer` 全是 `'\0'`，`%s` 显示为空字符串，屏幕上出现 100 个空格占位。

打印完成后把当前位置设为 `=`，下一帧就能多显示一个进度字符。

这个顺序是：

```
显示当前 frame
→ 填充下一格
→ 再次刷新
```

7.5 固定宽度格式

```
printf("[%-100s][%3d%%][%c]\r", buffer, count, label[...]);
```

- `%-100s`：左对齐，最小宽度 100。
- `%3d`：百分比至少占 3 个字符。
- `%%`：输出一个真正的百分号。
- `%c`：输出一个旋转动画字符。
- `\r`：回到行首。

7.6 第一版的局限

第一版的进度完全由自己的 `while` 循环控制。真实下载、复制或计算任务的进度来自业务逻辑，不能每次都在进度条函数内部写一个固定循环。

所以需要把“显示进度”和“产生进度”分开。

8. 第二版进度条：显示与业务分离

8.1 设计目标

第二版需要能服务于真实场景，例如：

```
下载文件
→ 根据已下载大小和总大小更新进度
```

进度条只负责显示，下载函数只负责提供 `current` 和 `total`。

8.2 进度百分比计算

```
百分比 = current / total × 100%
```

需要处理：

- `total` 不能为 0。
- 结果应限制在 0 到 100。
- 浮点数格式化时保留合适的小数位。

8.3 单独声明显示接口

```
// process.h
#pragma once

void FlushProcess(double total, double current);
```

该函数接收总量和当前进度，不负责执行下载任务。

8.4 实现显示函数

```
// process.c
#include "process.h"
#include <stdio.h>
#include <string.h>

#define NUM 101
#define STYLE '='

void FlushProcess(double total, double current)
{
    char buffer[NUM];
    const char *label = "|/-\\";
    static int spin_index = 0;
    int filled = 0;
    double percent = 0.0;

    memset(buffer, 0, sizeof(buffer));

    if (total > 0.0)
    {
        percent = current * 100.0 / total;
    }

    if (percent < 0.0)
        percent = 0.0;
    if (percent > 100.0)
        percent = 100.0;

    filled = (int)percent;

    for (int i = 0; i < filled; ++i)
        buffer[i] = STYLE;
```

```
    printf("[%-100s][%6.2f%%][%c]\r",
           buffer,
           percent,
           label[spin_index % 4]);

    fflush(stdout);
    ++spin_index;
}
```

8.5 使用静态旋转索引

```
static int spin_index = 0;
```

`static` 局部变量在函数调用之间保留值，因此每次刷新可以切换：

```
|
/
-
\
```

旋转动画依赖函数刷新次数，而不是固定写在主循环中。

8.6 主程序提供真实进度

```
// main.c
#include "process.h"
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    const double total = 1024.0;
    double current = 0.0;

    while (current <= total)
    {
        FlushProcess(total, current);
        usleep(3000);
        current += 1.0;
```

```
    }

    FlushProcess(total, total);
    printf("\n");

    return 0;
}
```

编译：

```
gcc main.c process.c -o processbar
```

运行：

```
./processbar
```

8.7 第二版为什么更适合真实项目

8.7.1 进度显示与业务分离

下载逻辑不需要知道 `printf`、缓冲区、旋转字符或回车符，只需要报告当前进度。

显示函数也不需要知道文件来自网络、磁盘还是内存。

8.7.2 代码可复用

只要提供 `current` 和 `total`，同一个显示函数就可以用于：

- 下载进度。
- 文件复制。
- 数据处理。
- 安装步骤。

8.7.3 更容易维护

如果要改颜色、宽度或动画，只需要修改显示函数，不必改动业务逻辑。

9. 回调函数与解耦

9.1 直接调用与回调

9.1.1 直接调用

```
FlushProcess(total, current);
```

业务函数直接知道具体进度显示函数。对简单项目已经足够。

9.1.2 使用函数指针

更通用的方式是给业务函数传入一个显示回调：

```
typedef void (*ProgressCallback)(double total, double current);
```

业务函数只关心：

```
现在进度是多少
→ 调用 callback(total, current)
```

9.2 回调示例

```
// process.h
#pragma once

typedef void (*ProgressCallback)(double total, double current);

void FlushProcess(double total, double current);
void Download(double total, ProgressCallback on_progress);
```

```
// process.c
#include "process.h"
#include <stddef.h>
#include <unistd.h>

void Download(double total, ProgressCallback on_progress)
```

```
{
    double current = 0.0;

    while (current <= total)
    {
        if (on_progress != NULL)
            on_progress(total, current);
        usleep(3000);
        current += 1.0;
    }

    if (on_progress != NULL)
        on_progress(total, total);
}
```

```
// main.c
#include "process.h"
#include <stdio.h>

int main(void)
{
    Download(1024.0, FlushProcess);
    printf("\n");
    return 0;
}
```

9.3 回调带来的好处

9.3.1 降低耦合

`Download` 不需要依赖某个具体的进度显示界面。理论上可以传入：

- 终端进度条。
- 图形界面更新函数。
- 日志记录函数。
- 测试用的假回调。

9.3.2 提高可测试性

测试下载逻辑时，可以传入一个只记录调用次数的回调，不必真的打印终端进度条。

9.3.3 保留上层控制

进度如何显示由调用者决定，业务函数只负责报告进度。

10. 边界情况与质量改进

10.1 百分比超界

浮点误差或异常输入可能让 `current` 略大于 `total`。显示前应将百分比限制在 100% 以内：

```
if (percent < 0.0)
    percent = 0.0;
if (percent > 100.0)
    percent = 100.0;
```

10.2 total 为 0

直接计算：

```
current / total
```

会发生除零或得到无穷大。显示函数应先判断：

```
if (total > 0.0)
    percent = current * 100.0 / total;
```

10.3 文本残留

如果每帧宽度不固定，旧帧的字符可能残留在末尾。解决方式：

- 使用固定宽度。
- 在输出末尾补空格。
- 必要时输出清行控制序列。

基础进度条优先使用固定宽度，不依赖复杂终端控制。

10.4 刷新过慢或过快

10.4.1 过慢

刷新太少会导致进度条一跳一跳，甚至程序结束时才显示。

10.4.2 过快

每毫秒刷新一次会占用大量终端输出，反而看不清，也可能降低程序性能。

应根据任务总时长选择合理刷新频率。

10.5 输出被重定向

进度条在终端中使用 `\r` 很自然，但把输出重定向到文件时，回车符和缓冲行为可能不符合用户预期。

更完整的程序可以检测是否连接终端，再决定：

```
终端 → 动态进度条
文件 → 定期打印百分比日志
```

10.6 中断与输出完整性

如果程序被中断，行尾可能还没有换行。可以在正常退出前统一补：

```
printf("\n");
```

这样下一条终端提示符不会覆盖进度条。

11. 完整实验流程

11.1 实验一：观察行缓冲

分别运行：

```c
printf("hello\n");
```

```c
printf("hello");
```

```c
printf("hello");
fflush(stdout);
```

三种代码都在后面加入：

```c
sleep(3);
```

记录输出出现的时机。

11.2 实验二：倒计时

使用：

```c
printf("count is: %-2d\r", count);
fflush(stdout);
sleep(1);
```

观察：

- 数字是否在原位置变化。
- 去掉 `\r` 后是否向下换行。
- 去掉 `fflush` 后是否延迟。

11.3 实验三：第一版进度条

1. 编译第一版代码。
2. 把 `usleep(50000)` 改大或改小。
3. 把进度区域从 100 改成其他宽度。
4. 去掉 `fflush(stdout)`，观察结果。
5. 把 `\r` 改成 `\n`，观察结果。

11.4 实验四：第二版模块化

创建：

```
process.h
process.c
main.c
```

编译：

```
gcc main.c process.c -o processbar
```

分别修改：

- 进度条宽度。
- 旋转光标字符。
- 百分比小数位。
- 刷新间隔。

观察业务逻辑是否需要跟着改。

11.5 实验五：回调接口

1. 把 `FlushProcess` 作为函数指针传给下载函数。
2. 再写一个只输出百分比的回调。
3. 替换回调验证下载逻辑不变。

12. 代码速查与常见错误

12.1 核心技术速查

技术	作用
`\r`	光标回到行首
`\n`	移动到下一行
`printf`	格式化文本输出
`fflush(stdout)`	立即刷新标准输出

技术	作用
`sleep(1)`	暂停约 1 秒
`usleep(50000)`	暂停约 0.05 秒
`%s`	输出字符串
`%-100s`	左对齐、宽度 100
`%%`	输出百分号字符
`static`	在函数调用间保留局部变量

12.2 常见错误

12.2.1 忘记 fflush

表现：

- 进度条不更新。
- 程序结束时才一次性出现。

12.2.2 使用 \n 代替 \r

表现：

- 每一帧都输出到新行。
- 没有覆盖效果。

12.2.3 数组越界

表现：

- 输出乱码。
- 程序不稳定。
- 不同运行环境表现不同。

12.2.4 百分比没有限制

表现：

- 超过 100%。
- 负进度。
- 显示区域长度异常。

12.2.5 total 为 0

表现：

- 除零。
- `nan` 或 `inf`。

12.2.6 业务逻辑写死在显示函数中

表现：

- 进度条只能用于某一个下载循环。
- 改任务时大量复制代码。

12.3 学习主线

```
理解 \r 与 \n
→ 理解 stdout 缓冲
→ 使用 fflush 强制刷新
→ 用倒计时验证行内覆盖
→ 实现固定宽度进度条
→ 把显示与业务拆开
→ 用回调进一步解耦
→ 处理边界情况和终端适配
```

本节只整理进度条实战相关内容，不展开 Git 或其他开发工具。